

Тема 8 Право доступа. Индексы и исключения. Каталог системы

Каждый пользователь в среде SQL имеет специальное идентификационное имя или номер. Терминология везде разная, но мы выбрали (следуя ANSI) ссылку на имя или номер как на *Идентификатор (ID)* доступа. Команда, посланная серверу базы данных, ассоциируется с определенным пользователем, или, иначе, специальным *Идентификатором доступа*. Поскольку это относится к SQL базе данных, ID разрешения — это имя пользователя, и SQL может использовать специальное ключевое слово **USER**, которое ссылается к Идентификатору доступа связанному с текущей командой. Команда интерпретируется и разрешается (или запрещается) на основе информации связанной с Идентификатором доступа пользователя подавшего команду.

Каждый пользователь в SQL базе данных имеет набор привилегий. Это – то, что пользователю разрешается делать. Эти привилегии могут изменяться со временем – новые добавляться, старые удаляться. Некоторые из этих привилегий определены в ANSI SQL, но имеются и дополнительные привилегии, которые являются также необходимыми.

Стандартные привилегии

SQL привилегии, определенные ANSI – это привилегии объекта. Это означает, что пользователь имеет привилегию, чтобы выполнить данную команду только на определенном объекте в базе данных. Очевидно, что привилегии должны различать эти объекты, но система привилегий, основанная исключительно на привилегиях объекта не может описывать все, что нужно SQL.

Привилегии объекта связаны одновременно и с пользователями, и с таблицами. То есть, привилегия дается определенному пользователю в указанной таблице, или базовой таблице или представлении. Надо помнить, что пользователь, создавший таблицу (любого вида), является владельцем этой таблицы. Это означает, что пользователь имеет все привилегии в этой таблице и может передавать привилегии другим пользователям в этой таблице.

Привилегии, которые можно назначить пользователю:

SELECT	- Пользователь с этой привилегией может выполнять запросы в таблице.
INSERT	- Пользователь может выполнять команду INSERT в таблице.
UPDATE	- Пользователь с этой привилегией может выполнять команду UPDATE на таблицы, а также ограничить эту привилегию для определенных столбцов таблицы.
DELETE	- Пользователь с этой привилегией может выполнять команду DELETE в таблице.

REFERENCES - Пользователь определить внешний ключ, который использует один или более столбцов этой таблицы, как родительский ключ. Вы можете ограничить эту привилегию для определенных столбцов.

Команда GRANT

Предположим, что пользователь Diane имеет таблицу Заказчиков и хочет позволить пользователю Adrian выполнить запрос к ней. Diane должна, в этом случае ввести следующую команду:

GRANT SELECT ON Customers TO Adrian;

Теперь Adrian может выполнить запросы к таблице Заказчиков. Без других привилегий, он может только выбрать значения; но не может выполнить любое действие, которые бы воздействовало на значения в таблице Заказчиков (включая использование таблицы Заказчиков в качестве родительской таблицы внешнего ключа, что ограничивает изменения, которые можно выполнять со значениями в таблице Заказчиков).

Когда SQL получает команду GRANT, он проверяет привилегии пользователя, подавшего эту команду, чтобы определить, допустима ли команда GRANT.

Adrian самостоятельно не может выдать эту команду. Он также не может предоставить право SELECT другому пользователю: таблица еще принадлежит Diane (позже мы покажем, как Diane может дать право Adrian предоставлять SELECT другим пользователям).

Синтаксис – тот же самый, что и для предоставления других привилегий. Если Adrian – владелец таблицы Продавцов, то он может позволить Diane вводить в нее строки с помощью следующего предложения:

GRANT INSERT ON Salespeople TO Diane;

Теперь Diane имеет право помещать нового продавца в таблицу.

Не следует ограничивать себя предоставлением одиночной привилегии отдельному пользователю командой GRANT. Списки привилегий или пользователей, отделяемых запятыми, являются совершенно приемлемыми. Stephen может предоставить и SELECT и INSERT в таблице Заказов для Adrian:

GRANT SELECT, INSERT ON Orders TO Adrian;

или и для Adrian и для Diane:

GRANT SELECT, INSERT ON Orders TO Adrian, Diane;

Когда привилегии и пользователи перечислены таким образом, весь список привилегий предоставляются всем указанным пользователям.

Управление всеми привилегиями объекта использует один тот же синтаксис, кроме привилегий на выполнение команд UPDATE и REFERENCES, для которых можно указывать имена столбцов. Привилегию UPDATE можно предоставлять наподобие других привилегий:

GRANT UPDATE ON Salespeople TO Diane;

Эта команда позволит Diane изменять значения в любом или во всех столбцах таблицы Продавцов. Однако, если Adrian хочет ограничить Diane в изменении например комиссионных, он может ввести

GRANT UPDATE (comm) ON Salespeople TO Diane;

Другими словами, он просто должен указать конкретный столбец, к которому привилегия UPDATE должна быть применена, в круглых скобках после имени таблицы. Имена нескольких столбцов таблицы могут указываться в любом порядке, отделяемые запятыми:

GRANT UPDATE (city, comm) ON Salespeople TO Diane;

REFERENCES следует тому же самому правилу. Когда вы предоставите привилегию REFERENCES другому пользователю, он сможет создавать внешние ключи ссылающиеся на столбцы вашей таблицы как на родительские ключи. Подобно UPDATE, для привилегии REFERENCES может быть указан список из одного или более столбцов, для которых ограничена эта привилегия. Например, Diane может предоставить Stephen право использовать таблицу Заказчиков, как таблицу родительского ключа, с помощью такой команды:

GRANT REFERENCES (cname, cnum) ON Customers TO Stephen;

Эта команда дает Stephen право использовать столбцы cnum и cname, в качестве родительских ключей по отношению к любым внешним ключам в его таблицах. Stephen может контролировать то, как это будет выполнено. Он может определить (cname, cnum) или, как в нашем случае (cnum, cname), как двухстолбцовый родительский ключ, совпадающий с помощью внешнего ключа с двумя столбцами в одной из его собственных таблиц. Или он может создать отдельные внешние ключи, чтобы ссылаться на поля индивидуально, обеспечив тем самым, чтобы Diane имела принудительное присвоение родительского ключа.

Не имея ограничений на номера внешних ключей, он должен базироваться на этих родительских ключах, а родительские ключи различных внешних ключей разрешены для совмещения (overlap).

Как и в случае с привилегией UPDATE, вы можете исключить список столбцов и таким образом позволять всем без исключения столбцам быть используемыми в качестве родительских ключей. Adrian может предоставить Diane право сделать это следующей командой:

GRANT REFERENCES ON Salespeople TO Diane;

Привилегия будет пригодна для использования только в столбцах, которые имеют ограничения, требуемые для родительских ключей.

SQL поддерживает два аргумента для команды GRANT, которые имеют специальное значение: ALL PRIVILEGES (ВСЕ ПРИВИЛЕГИИ) или просто ALL и PUBLIC (ОБЩИЕ). ALL используется вместо имен привилегий в команде GRANT, чтобы отдать все привилегии в таблице. Например, Diane может дать Stephen весь набор привилегий в таблице Заказчиков с помощью такой команды:

GRANT ALL ON Customers TO Stephen;

(привилегии UPDATE и REFERENCES применяются ко всем столбцам.)

Когда вы предоставляете привилегии PUBLIC, все пользователи автоматически их получают. Наиболее часто, это применяется для привилегии SELECT в определенных базовых таблицах или представлениях, которые вы хотите сделать доступными для любого пользователя. Чтобы позволить любому пользователю видеть таблицу Заказов, вы, например, можете ввести следующее:

GRANT SELECT ON Orders TO PUBLIC;

Модификатор PUBLIC не ограничен в его передаче только текущим пользователям. Любой новый пользователь, добавляемый к вашей системе, автоматически получит все привилегии, назначенные ранее всем, так что если вы захотите ограничить доступ к таблице всем, сейчас или в будущем, лучше всего предоставить привилегии, иные, чем SELECT для индивидуальных пользователей.

Иногда создателю таблицы необходимо, чтобы другие пользователи могли получить привилегии в его таблице. Обычно это делается в системах, где один или более людей создают несколько (или все) базовые таблицы в базе данных, а затем передают ответственность за них тем, кто будет фактически с ними работать. SQL позволяет делать это с помощью предложения WITH GRANT OPTION.

Если Diane хотела бы, чтобы Adrian имел право предоставлять привилегию SELECT в таблице Заказчиков другим пользователям, она дала бы ему привилегию SELECT с использованием предложения WITH GRANT OPTION:

GRANT SELECT ON Customers TO Adrian WITH GRANT OPTION;

После того Adrian получил право передавать привилегию SELECT третьим лицам. Он может выдать команду

GRANT SELECT ON Diane.Customers TO Stephen;

или даже

GRANT SELECT ON Diane.Customers TO Stephen WITH GRANT OPTION;

Пользователь с помощью GRANT OPTION в особой привилегии для данной таблицы, может, в свою очередь, предоставить эту привилегию к той же таблице, с или без GRANT OPTION, любому другому пользователю. Это не меняет принадлежности самой таблицы; как и прежде таблица принадлежат ее создателю. Пользователь же с помощью GRANT OPTION во всех привилегиях для данной таблицы будет иметь всю полноту власти в той таблице.

Потребность удалять привилегии сводится к команде REVOKE, фактически стандартному средству с достаточно понятной формой записи.

Синтаксис команды REVOKE – похож на GRANT, но имеет обратный смысл. Чтобы удалить привилегию INSERT для Adrian в таблице Заказов, вы можете ввести

REVOKE INSERT ON Orders FROM Adrian;

Использование списков привилегий и пользователей здесь допустимо, как и в случае с GRANT, так что можно ввести следующую команду:

REVOKE INSERT, DELETE ON Customers FROM Adrian, Stephen;

Привилегии отменяются пользователем, который их предоставил, и отмена будет каскадироваться, то есть она будет автоматически распространяться на всех пользователей, получивших от него эту привилегию.

Вы можете сделать действия привилегий более точными, используя представления. Всякий раз, когда вы передаете привилегию в базовой таблице пользователю, она автоматически распространяется на все строки, а при использовании возможных исключений UPDATE и REFERENCES, на все столбцы таблицы.

Создавая представление, которое ссылается на основную таблицу, и затем, перенося привилегию на представление, а не на таблицу, вы можете ограничивать эти привилегии любыми выражениями в запросе, содержащимся в представлении. Это значительно улучшает базисные возможности команды GRANT.

Чтобы создавать представление, вы должны иметь привилегию SELECT во всех таблицах, на которые вы ссылаетесь в представлении. Если представление – модифицируемое, любая привилегия INSERT, UPDATE и DELETE, которые вы имеете в базовой таблице, будут автоматически передаваться представлению.

Предположим, вы хотите дать пользователю Claire способность видеть только столбцы snum и sname таблицы Продавцов. Вы можете сделать это, поместив имена этих столбцов в представление

```
CREATE VIEW Claairesview  
AS SELECT snum, sname  
FROM Salespeople;
```

и предоставив Claire привилегию SELECT в представлении, а не в самой таблице Продавцов:

```
GRANT SELECT On Claairesview to Claire;
```

Можно создать привилегии специально для столбцов наподобие использования других привилегий, но для команды INSERT это будет означать вставку значений по умолчанию, а для команды DELETE, ограничение столбца не будет иметь значения. Привилегии REFERENCES и UPDATE, конечно, могут сделать столбцы специфическими, не прибегая к представлению.

Чтобы привилегия относилась только к определенным строкам, надо использовать предикат в представлении, который определит, какие строки являются включенными. Например, чтобы предоставить пользователю Adrian привилегию UPDATE в таблице Заказчики для всех заказчиков, размещенных в Лондоне, можно создать представление:

```
CREATE VIEW Londoncust  
AS SELECT *  
FROM Customers  
WHERE city = 'London'  
WITH CHECK OPTION;
```

Затем надо передать привилегию UPDATE в этом представлении для Adrian:

```
GRANT UPDATE ON Londoncust TO Adrian;
```

В этом отличие привилегии для определенных строк от привилегии UPDATE для определенных столбцов, которая распространена на все столбцы таблицы Заказчики, но не на строки, среди которых строки со значением поля city иным, чем London, не будут учитываться. Предложение WITH CHECK OPTION предохраняет Adrian от замены значения поля city на любое значение кроме London.

Другая возможность состоит в том, чтобы дать пользователям доступ к уже извлеченным данным, а не к фактическим значениям в таблице. Агрегатные функции могут быть весьма удобными в применении такого способа. Например, создать представление, которое определяет общую стоимость и среднее значение покупок и их общее количество для Заказов на каждый день:

```
CREATE VIEW Datetotals  
AS SELECT odate, COUNT (*), SUM (amt), AVG (amt)  
FROM Orders  
GROUP BY odate;
```

Далее надо передать пользователю Diane привилегию SELECT в представлении Datetotals:

```
GRANT SELECT ON Datetotals TO Diane;
```

Предположим, что необходимо удостовериться, что все значения поля city в таблице Продавцов находятся в одном из городов, где ваша компания в настоящее время имеет ведомство. Вы можете установить ограничение CHECK непосредственно на столбец city, но позже может стать трудно его изменить, если ваша компания, например, откроет там другие ведомства. В качестве альтернативы, можно создать представление, которое исключает неправильные значения city:

```
CREATE VIEW Curcities  
AS SELECT *  
FROM Salespeople  
WHERE city IN ('London', 'Rome', 'San Jose', 'Berlin')  
WITH CHECK OPTION;
```

Теперь, вместо того, чтобы предоставить пользователям привилегии модифицирования в таблице Продавцы, вы можете предоставить их в представлении Curcities. Преимущество такого подхода – в том, что если вам нужно сделать изменение, то можно удалить это представление, создав новое, и предоставить в этом новом представлении привилегии пользователям, что проще, чем изменять ограничения. Недостатком является то, что владелец таблицы Продавцов также должен использовать это представление, если он не хочет, чтобы его собственные команды были отклонены.

С другой стороны, этот подход позволяет владельцу таблицы и любым другим получить привилегии модификации в самой таблице, а не в представлении, чтобы делать исключения для ограничений. Это часто бывает желательно, но не выполнимо, если вы используете ограничения в базовой таблице. К сожалению, эти исключения нельзя будет увидеть в представлении. Если вы выберете этот подход, то захочется создать второе представление, содержащее только исключения:

```
CREATE VIEW Othercities  
AS SELECT *  
FROM Salespeople  
WHERE city NOT IN ('London', 'Rome', 'San Jose', 'Berlin')  
WITH CHECK OPTION;
```

Вы должны выбрать для передачи пользователям только привилегию SELECT в этом представлении, чтобы они могли видеть исключенные строки, но не могли помещать недопустимые значения city в базовую таблицу. Фактически, пользователи могли бы сделать запрос обоих представлений в объединении и увидеть все строки сразу.

Другие типы привилегий

Кто же имеет право первым создать таблицу? Кто имеет право изменять, удалять, или ограничивать таблицы? Должны ли права создания базовых таблиц отличаться от прав создания представлений? Должна ли система иметь суперпользователя – пользователя, который отвечает за поддержание базы данных и, следовательно, имеющий наибольшие, или все привилегии, которые не предоставляются индивидуально?

Привилегии, которые не определяются в терминах специальных объектов данных, называются привилегиями системы, или правами базы данных. На базисном уровне они будут, вероятно, включать в себя право создавать объекты данных, вероятно, отличающиеся от базовых таблиц (обычно создаваемых несколькими пользователями) и представления (обычно создаваемые большинством пользователей).

Кроме того, в системе любого размера всегда имеются некоторые типы суперпользователей – пользователей, которые автоматически имеют большинство или все привилегии, — и которые могут передать свой статус суперпользователя кому-нибудь с помощью привилегии или группы привилегий.

Типичные привилегии системы

При общем подходе имеется три базовых привилегии системы:

- CONNECT (Подключить),
- RESOURCE (Ресурс),
- DBA (Администратор Базы Данных).

Проще, можно сказать, что CONNECT состоит из права зарегистрироваться и права создавать представления и синонимы, если переданы привилегии объекта. RESOURCE состоит из права создавать базовые таблицы. DBA – это привилегия суперпользователя, дающая пользователю высокие полномочия в базе данных. Один или более пользователей с функциями администратора базы данных может иметь эту привилегию. Фактически только один человек имеет право зарегистрироваться с именем SYSADM, являющимся его идентификатором доступа. Команда GRANT в измененной форме является пригодной для использования с привилегиями объекта, как и с системными привилегиями. Для начала передача прав может быть сделана с помощью DBA. Например, DBA может передать привилегию для создания таблицы пользователю Rodriguez следующим образом:

GRANT RESOURCE TO Rodriguez;

Естественно, появляется вопрос, откуда возьмется пользователь с именем Rodriguez? Как определить его ID допуска? В большинстве реализаций, DBA создает пользователя, автоматически предоставляя ему привилегию CONNECT.

В этом случае, обычно добавляется предложение IDENTIFIED BY, указывающее пароль. Если же нет, операционная система должна определить, можете ли вы зарегистрироваться в базе данных с данным ID доступа. DBA может, например, ввести

GRANT CONNECT TO Thelonius IDENTIFIED BY Redwagon;

что приведет к созданию пользователя с именем Thelonius, даст ему право регистрироваться, и назначит ему пароль Redwagon, и все это в одном предложении.

Раз Thelonius – опознанный пользователь, то он или DBA могут использовать эту же команду чтобы изменить пароль Redwagon. Если вы хотите запретить пользователю регистрироваться, вы должны использовать для REVOKE привилегию CONNECT, которая "удаляет" этого пользователя. Некоторые реализации позволяют вам создавать и удалять пользователей, независимо от их привилегий при регистрации.

Когда вы предоставляете привилегию CONNECT пользователю, вы создаете этого пользователя. При этом чтобы сделать это, Вы сами должны иметь DBA привилегию. Если этот пользователь будет создавать базовые таблицы (а не только представления), ему нужно также предоставить привилегию RESOURCE. Но это сразу порождает другую проблему.

Если вы сделаете попытку удалить привилегию CONNECT пользователя, который имеет им созданные таблицы, команда будет отклонена, потому что ее действие оставит таблицу без владельца, а это не допускается. Для этого надо сначала удалить все таблицы, созданные этим пользователем, прежде чем удалить его привилегию CONNECT. Если эти таблицы не пустые, то вы, вероятно, захотите передать их данные в другие таблицы с помощью команды INSERT, которая использует запрос. Вам не нужно удалять отдельно привилегию RESOURCE; достаточно удалить CONNECT, чтобы удалить пользователя.

Индекс

Индекс – это упорядоченный (буквенный или числовой) список столбцов или групп столбцов в таблице

Таблицы могут иметь большое количество строк, и не находятся в каком-нибудь определенном порядке, их поиск по указанному значению может потребовать значительного времени

Когда создается индекс в поле, БД запоминает соответствующий порядок всех значений этого поля в области памяти.

Например, таблица Заказчики имеет тысячи входов, а вы хотите найти заказчика с номером snum=2999

Т.к. строки не упорядочены, программа будет просматривать всю таблицу, строку за строкой, проверяя каждый раз значение поля snum на равенство значению 2999.

Однако если бы имелся индекс в поле snum, то программа могла бы выйти на номер 2999 прямо по индексу и дать информацию о том, как найти правильную строку таблицы

Индексы значительно повышают эффективность запросов.

Однако, использование индекса несколько замедляет операции модификации **INSERT**, **UPDATE** и **DELETE**

Почему? При модификации таблицы модифицируется и индекс, который занимает некоторый объем памяти. Поэтому, каждый раз, когда создается таблица, надо принять решение, индексировать ее или нет.

Индексы могут состоять из многочисленных полей.

Синтаксис для создания индекса:

```
CREATE INDEX <index name>  
ON <table name>  
(<column name>  
    [,<column name>]...);
```

Таблица, должна быть создана и содержать имена столбцов

Имя индекса не может быть использовано для чего-то другого в базе данных.

Например, если таблица Заказчики будет наиболее часто упоминаемой в запросах продавцов к их собственной клиентуре, то можно создать индекс в поле snum таблицы Заказчиков.

```
CREATE INDEX Clientgroup  
ON Customers (snum);
```

Теперь, продавец, который имеет отношение к этой таблице, сможет найти собственную клиентуру достаточно быстро.

Лучше всего создавать индексы сразу после того, как таблица создана и прежде, чем введены любые значения.

Удаление индексов

Главным признаком индекса является его имя, поэтому он может быть удален.

Обычно пользователи не знают о существовании индекса.

SQL автоматически определяет, позволено ли пользователю использовать индекс, и если да, то разрешает использовать его.

Однако, если надо удалить индекс, то необходимо знать его имя.

Следующий синтаксис используется для удаления индекса:

```
DROP INDEX <Index name>;
```

Удаление индекса не воздействует на содержание полей.

Каталог системы

Чтобы база данных функционировала, компьютерная система должна следить за таблицами, представлениями, индексами, синонимами, привилегиями, пользователями, и т.д. Для этого существуют системный каталог, словарь данных, или просто системные таблицы.

Таблицы системного каталога – напоминают обычные SQL таблицы: те же строки и столбцы данных. Например, одна таблица каталога обычно содержит информацию о таблицах, существующих в БД, другая – содержит информацию о различных столбцах таблиц и т.д.. Таблицы каталога создаются и присваиваются с помощью самой базы данных и идентифицируются с помощью специальных имен, таких, например, как **SYSTEM**. База данных создает эти таблицы и модифицирует их автоматически, таблицы каталога не могут быть непосредственно подвергнуты действию команды модификации.

В большинстве систем, каталог может быть запрошен пользователем. Это очень полезно, т.к. это дает нам возможность узнать кое-что о БД, которая используется. Вся информация не всегда доступна всем пользователям, доступ к каталогу ограничен для пользователей без соответствующих привилегий.

Обычно, привилегии каталога предоставляет суперпользователь, например, администратор системы, зарегистрированный как **SYSTEM** или **DBA**. Кроме того, некоторые привилегии могут предоставляться пользователям автоматически.

Рассмотрим некоторые таблицы, которые можно найти в типовом каталоге системы:

- **SYSTEMCATALOG** Таблицы (базовые и представления)
- **SYSTEMCOLUMNS** Столбцы таблицы
- **SYSTEMTABLES** Каталог Представления в **SYSTEMCATALOG**
- **SYSTEMINDEXES** Индексы в таблице
- **SYSTEMUSERAUTH** Пользователи базы данных
- **SYSTEMTABAUTH** Объектные привилегии пользователей
- **SYSTEMCOLAUTH** Столбцовые привилегии пользователей
- **SYSTEMSYNONS** Синонимы для таблиц

Если **DBA** предоставит пользователю Stephen право просматривать **SYSTEMCATALOG**, то такой командой будет следующая:

GRANT SELECT ON SYSTEMCATALOG TO Stephen;

то Stephen сможет увидеть некоторую информацию о всех БД

Поскольку **SYSTEMCATALOG** – это таблица, то можно использовать ее в представлении. Фактически можно считать, что имеется такое представление с именем **SYSTEMTABLES**.

Представление **SYSTEMCATALOG** содержит только те таблицы, которые входят в системный каталог.

Предположим, что только таблицы каталога являются собственностью пользователя **SYSTEM**. Можно определить другое представление, которое бы специально исключало таблицы каталога из вывода:

```
CREATE VIEW Datatables  
AS SELECT *  
FROM SYSTEMCATALOG  
WHERE owner <> 'SYSTEM';
```

Кроме этого, имеются другое использование представлений каталога. Предположим нужно, чтобы каждый пользователь был способен сделать запрос каталога, для получения информации только из таблиц, которыми он владеет. Пока значение **USER**, в команде SQL постоянно для ID доступа пользователя выдающего команду, оно может всегда быть использоваться, чтобы давать доступ пользователям только к их собственным таблицам.

Можно создать следующее представление:


```
CREATE VIEW Owntables
AS SELECT *
FROM SYSTEMCATALOG
WHERE Owner = USER;
```

Теперь можно предоставить всем пользователям доступ к этому представлению:

```
GRANT SELECT ON Owntables TO PUBLIC;
```

Каждый пользователь теперь способен выбирать (SELECT) только те строки из SYSTEMCATALOG, владельцем которых он сам является.

ПРЕДСТАВЛЕНИЕ SYSTEMCOLUMNS. Одно небольшое добавление к этому позволит каждому пользователю просматривать таблицу SYSTEMCOLUMNS для столбцов из его собственных таблиц. Сначала, давайте рассмотрим ту часть таблицы SYSTEMCOLUMNS, которая описывает наши типовые таблицы (другими словами, исключим сам каталог):

<u>tname</u>	<u>cname</u>	<u>datatype</u>	<u>cnumber</u>	<u>tabowner</u>
Salespeople	snum	integer	1	Diane
Salespeople	sname	char	2	Diane
Salespeople	city	char	3	Diane
Salespeople	comm	decimal	4	Diane
Customers	cnum	integer	1	Claire
Customers	cname	char	2	Claire
Customers	city	char	3	Claire
Customers	rating	integer	4	Claire
Customers	snum	integer	5	Claire
Orders	onum	integer	1	Diane
Orders	odate	date	2	Diane
Orders	amt	decimal	3	Diane
Orders	cnum	integer	4	Diane
Orders	snum	integer	5	Diane

Каждая строка этой таблицы показывает столбец таблицы в базе данных. Все столбцы данной таблицы должны иметь разные имена, также как каждая таблица должна иметь данного пользователя, и наконец все комбинации пользователей, таблиц, и имен столбцов должны различаться между собой.

Следовательно, табличные столбцы: tname (имя таблицы), tabowner (владелец таблицы), и sname (имя столбца), вместе составляют первичный ключ этой таблицы. Столбец datatype (тип данных) говорит сам за себя. Столбец cnumber (номер столбца) указывает на местоположение этого столбца внутри таблицы. Для упрощения, мы опустили параметры длины столбца, точности, и масштаба.

Некоторые SQL реализации будут обеспечивать вас большим количеством данных, чем показано в этих столбцах, но показанные являются основой для любой реализации.

Пример, позволяющий каждому пользователю видеть информацию SYSTEMCOLUMNS только для принадлежащих ему таблиц:

```
CREATE VIEW Owncolumns
AS SELECT *
FROM SYSTEMCOLUMNS
WHERE tabowner = USER;
```

```
GRANT SELECT ON Owncolumns TO PUBLIC;
```

Большинство версий SQL позволяют вам помещать комментарии (ремарки) в специальные столбцы пояснений таблиц каталогов SYSTEMCATALOG и SYSTEMCOLUMNS, что удобно, так как эти таблицы не всегда могут объяснить свое содержание.

Можно использовать команду COMMENT ON со строкой текста, чтобы пояснить любую строку в одной из этих таблиц. Состояние -TABLE, для комментирования в SYSTEMCATALOG, и текст – COLUMN, для SYSTEMCOLUMNS.

Например:

```
COMMENT ON TABLE Chris.Orders  
IS 'Current Customer Orders';
```

Текст будет помещен в столбец пояснений SYSTEMCATALOG. Обычно, максимальная длина таких пояснений – 254 символа.

Сам комментарий, указывается для конкретной строки, одна с tname=Orders, а другая owner=Chris. Можно увидеть этот комментарий в строке для таблицы Заказы в SYSTEMCATALOG:

```
SELECT tname, remarks  
FROM SYSTEMCATALOG  
WHERE tname = 'Orders' AND owner = 'Chris';
```

SYSTEMCOLUMNS работает точно так же. Чтобы изменить комментарий, надо ввести новую команду COMMENT ON для той же строки. Новый комментарий будет записан поверх старого. Если вы хотите удалить комментарий, напишите поверх него пустой комментарий, подобно следующему:

```
COMMENT ON COLUMN Orders.onum  
IS '';
```

и этот пустой комментарий затрет предыдущий.

Покажем оставшиеся из ваших системных таблиц определения, с типовым запросом для каждого:

SYSTEMINDEXES – ИНДЕКСАЦИЯ В БАЗЕ ДАННЫХ

Имена столбцов в таблице SYSTEMINDEXES и их описания – следующие:

<u>СТОЛБЦЫ</u>	<u>ОПИСАНИЕ</u>
iname	Имя индекса (используемого для его исключения)
iowner	Имя пользователя который создал индекс
tname	Имя таблицы которая содержит индекс
cnumber	Номер столбца в таблице
tabowner	Пользователь который владеет таблицей содержащей индекс
numcolumns	Число столбцов в индексе
cposition	Позиция текущего столбца среди набора индексов
isunique	Уникален ли индекс (Y или N)

Все пользователи получают CONNECT по умолчанию при регистрации, поэтому он не описан в таблице выше. Возможные состояния столбцов resource и dba могут быть – Y (Да, пользователь имеет привилегии) или – No (пользователь не имеет привилегий).

Пароли (password) доступны только высоко привилегированным пользователям, если они существуют. Следовательно, запрос этой таблицы можно вообще делать только для информации относительно привилегий системы и пользователей.

Чтобы найти всех пользователей, которые имеют привилегию RESOURCE, и увидеть какие из них – DBA, можно ввести следующее условие:

```
SELECT username, dba  
FROM SYSTEMUSERAUTH  
WHERE resource = 'Y';
```

Вывод для этого запроса содержит имена столбцов в таблице SYSTEMTABAUTH и их описание:

<u>COLUMN</u>	<u>ОПИСАНИЕ</u>
username	Пользователь, который имеет привилегии
grantor	Пользователь, который передает привилегии по имени пользователя
tname	Имя таблицы, в которой существуют привилегии

owner	Владелец tname
selauth	Имеет ли пользователь привилегию SELECT
insauth	Имеет ли пользователь привилегию INSERT
delauth	Имеет ли пользователь привилегию DELETE

Возможные значения для каждой из перечисленных привилегий объекта (имена столбцов которых оканчиваются на auth) – Y, N, и G. G указывает, что пользователь имеет привилегию с возможностью передачи привилегий. В каждой строке, по крайней мере один из этих столбцов должен иметь состояние отличное от N (другими словами, иметь хоть какую-то привилегию).

Первые четыре столбца этой таблицы составляют первичный ключ. Это означает что каждая комбинация из tname – владелец-пользователь (две различные таблицы с различными владельцами могут иметь одно и тоже имя), пользователь и пользователь передающий права, должна быть уникальной. Каждая строка этой таблицы содержит все привилегии, предоставляются одним определенным пользователем другому определенному пользователю в конкретном объекте.

UPDATE и REFERENCES являются привилегиями, которые могут быть определенными столбцами, и находиться в различных таблицах каталога. Если пользователь получает привилегии в таблице от, чем одного пользователя, такие привилегии могут быть отдельными строками, созданными в этой таблице. Это необходимо для каскадного отслеживания при вызове привилегий.

Например, чтобы найти все привилегии SELECT, INSERT и DELETE, которые Adrian предоставляет пользователям в таблице Заказчики, можно ввести сделать запрос:

```
SELECT username, selauth, insauth, delauth
FROM SYSTEMTABAUTH
WHERE grantor = 'Adrian' AND tname = 'Customers';
```

Чтобы выяснить, в каких столбцах таблицы вы имеете привилегию REFERENCES, вы можете ввести следующий запрос:

```
SELECT owner, tname, cname
FROM SYSTEMCOLAUTH
WHERE refauth IN ('Y', 'G') AND username = USER
ORDER BY 1, 2;
```

Предположим, что Adrian имеет синоним Clients для таблицы Заказчиков, принадлежащей Diane, и что имеется общий синоним Customers для этой же таблицы. Можно делать запрос для всех синонимов в таблице Заказчики:

```
SELECT *
FROM SYSTEMSYNONS
WHERE tname = 'Customers'
```

Другое использование каталога

Конечно, можно выполнять более сложные запросы в системном каталоге. Объединения, например, могут быть очень удобны. Например, надо увидеть столбцы таблиц и базовые индексы, установленные для каждого:

```
SELECT a.tname, a.cname, iname, cposition
FROM SYSTEMCOLUMNS a, SYSTEMINDEXES b
WHERE a.tabowner = b.tabowner AND a.tname = b.tname AND
a.cnumber = b.cnumber
ORDER BY 3 DESC, 2;
```

Она показывает два индекса, один для таблицы Заказчиков и один для таблицы Продавцов. Последний из них – это одностолбцовый индекс с именем salesno в поле snum; он был помещен первым из-за сортировки по убыванию (в обратном алфавитном Заказе) в столбце iname. Другой индекс, custsale, используется продавцами, чтобы отыскивать своих заказчиков. Он основывается на комбинации полей snum и spnum внутри таблицы Заказчиков, с полем snum приходящим в индексе первым, как это и показано с помощью поля cposition.

tname	cname	iname	cposition
Salespeople	sname	salesno	1
Customers	cnum	custsale	2
Customers	snum	custsale	1

Подзапросы также могут быть использованы. Имеется способ увидеть данные столбца только для столбцов из таблиц каталога:

```
SELECT *
FROM SYSTEMCOLUMNS
WHERE tname IN (SELECT tname
FROM SYSTEMCATALOG);
```

Для простоты, мы не будем показывать вывод этой команды, которая состоит из одного входа для каждого столбца каждой таблицы каталога. Вы могли бы поместить этот запрос в представление, назвав его, например, SYSTEMTABCOLS, для представления SYSTEMTABLES.